“Multi-Process Command Pipeline System”

A C-Based Linux Application for Multi-Process Command Execution and Inter-Process Communication

OS & System Programming (OSSP) Hackathon – Project Report

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

By

SINDHUJA	**2520080045**
JHANASRI THOTA	**2520090121**
ALEKHYA	**2520090126**

Under the Esteemed Guidance of

[Guide Name]
[Designation]
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



Declaration

The Project Report entitled **"Multi-Process Command Pipeline System"** is a record of Bonafide work of **SINDHUJA – 2520080045, JHANASRI THOTA – 2520090121, ALEKHYA – 2520090126** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering to the K L (Deemed to be) University. The results embodied in this report have not been copied from any other departments/University/Institute.

Date: [dd-mm-yyyy] Place: [Hyderabad / campus]

(Signatures of the students)

_____ **SINDHUJA 2520080045**

_____ **JHANASRI THOTA 2520090121**

_____ **ALEKHYA 2520090126**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

This is to certify that the project report entitled **"Multi-Process Command Pipeline System"** is a bonafide work done and submitted by **Sindhuja – 2520080045, Jhanasri Thota – 2520090121 and Alekhya – 2520090126** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in the Department of Computer Science and Engineering, K L (Deemed to be University), during the academic year [20XX–20XX].

Signature of the Guide	**Signature of the Course Coordinator**	**Signature of the HOD**
[Guide name, designation]	[Coordinator name]	[HOD name]

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator [Course Coordinator name] and our guide [Guide name], Department of Computer Science and Engineering, for their tremendous support, encouragement and guidance throughout this project.

We express our gratitude to [HOD name], Head of the Department of Computer Science and Engineering, for providing us with adequate facilities, ways and means by which we were able to complete this project-based work.

We thank the organizers of the OS & System Programming Hackathon for providing a platform to apply operating-system concepts to a practical problem. We thank all the members of the teaching and non-teaching staff who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

SINDHUJA 2520080045

JHANASRI THOTA 2520090121

ALEKHYA 2520090126

ABSTRACT

Linux users routinely connect commands so that the output of one program becomes the input of the next. The operating-system work that makes this possible – creating processes, building pipes, redirecting file descriptors, executing programs and synchronizing their termination – is normally hidden inside the shell. This project, the **Multi-Process Command Pipeline System**, developed for the OS & System Programming Hackathon, implements that machinery explicitly as a C application for Linux.

The system reads a pipeline entered by the user, parses it into commands and arguments, and executes every command in its own child process. For N commands it creates N child processes and N − 1 pipes. Each child connects its standard input and output to the appropriate pipe ends using `dup2()`, closes unused descriptors with `close()` and replaces itself with the requested program using `execvp()`, while the parent process uses `waitpid()` to wait for all children and collect their exit statuses. On completion the application reports process IDs, exit statuses and elapsed time.

The project is built with C, GCC, GNU Make, Ubuntu Linux and Git/GitHub. A test suite covering basic execution, sorting, word counting, log analysis and invalid-command handling is extended in this report with boundary, error-handling and system-level checks. The system is a focused pipeline executor and not a complete shell, but it demonstrates process creation, inter-process communication, input/output redirection, command execution and process synchronization in a practical and observable form, and provides a foundation for learning Linux process management and for building lightweight command-orchestration tools.

Keywords: Linux, process management, pipe, fork, dup2, execvp, waitpid, inter-process communication.

INDEX

S.No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		**i**
	Abstract		**ii**
	Index		**iii**
	List of Figures		**vi**
	List of Tables		**vii**
1	**Introduction**		**1–4**
		1.1 Background of the Project	1
		1.2 Problem Statement	1
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	2
		1.5 Importance of the Application	3
		1.6 Target Users / Audience	3
		1.7 Organization of the Report	3
2	**OS Concepts and Background Study**		**5–8**
		2.1 Operating System Concepts Related to the Project	5
		2.2 Existing Methods / Algorithms	7
		2.3 Proposed Approach	7
		2.4 Advantages of the Proposed Approach	7
		2.5 Novelty and Innovation of the Project	8
3	**System Requirements**		**9–10**
		3.1 Hardware Requirements	9
		3.2 Software Requirements	9
		3.3 Operating System Environment – Ubuntu / WSL	9
		3.4 Development Tools and Compilers	10
		3.5 Functional Requirements	10
		3.6 Non-Functional Requirements	10
4	**Technology and Tools Used**		**11–13**
		4.1 C Programming Language	11
		4.2 GCC Compiler	11
		4.3 Linux System Calls	11
		4.4 POSIX Threads / Synchronization	12
		4.5 Linux Commands and Utilities	12
		4.6 Other Tools / Libraries Used	13
5	**System Architecture and Design**		**14–19**
		5.1 Overall System Architecture	14
		5.2 System Workflow	15
		5.3 Flowchart / Data Flow	16
		5.4 Module Description	18

S.No.	Chapter	Topics / Subtopics	Page No.
		5.5 Input and Output Design	18
		5.6 Design Decisions and Rationale	19
6	**Implementation**		**20–28**
		6.1 Module-wise Implementation	20
		6.2 Algorithm / Pseudocode	21
		6.3 Core C / C++ Implementation	22
		6.4 Linux System Calls / APIs Used	24
		6.5 Process / Thread Implementation	27
		6.6 IPC / Synchronization Implementation	27
		6.7 Memory Management Implementation	27
		6.8 Optional User Interface Implementation	28
		6.9 Error Handling Implementation	28
7	**Execution and Demonstration**		**29–32**
		7.1 Compilation in Ubuntu	29
		7.2 Execution Commands	29
		7.3 Sample Inputs and Outputs	29
		7.4 Screenshots of Execution	30
		7.5 Demonstration of OS Concepts	31
8	**Testing and Validation**		**33–36**
		8.1 Testing Strategy	33
		8.2 Test Environment	33
		8.3 Functional Test Cases	33
		8.4 Boundary and Error-Handling Test Cases	34
		8.5 System-Level (OS) Verification Tests	35
		8.6 Automated Test Harness	35
		8.7 Test Summary	36
		8.8 Traceability: Objectives to Tests	36
		8.9 Defect Log	36
9	**Results, Limitations, Future Scope and Business Impact**		**37–40**
		9.1 Results Achieved	37
		9.2 Measured Results	37
		9.3 Comparison with Alternative Approaches	37
		9.4 Limitations	38
		9.5 Future Scope	38
		9.6 Business Impact	38
10	**Hackathon Evaluation Criteria Mapping**		**41–44**
		10.1 Criterion 1 – Problem Definition (10 Marks)	42
		10.2 Criterion 2 – Novelty and Innovation (15 Marks)	42

S.No.	Chapter	Topics / Subtopics	Page No.
		10.3 Criterion 3 – Architecture Design (10 Marks)	42
		10.4 Criterion 4 – Implementation (20 Marks)	42
		10.5 Criterion 5 – Testing (10 Marks)	43
		10.6 Criterion 6 – Documentation (10 Marks)	43
		10.7 Criterion 7 – GitHub Repository (5 Marks)	43
		10.8 Criterion 8 – Presentation (10 Marks)	43
		10.9 Criterion 9 – Teamwork (5 Marks)	44
		10.10 Criterion 10 – Business Impact (5 Marks)	44
		10.11 Pre-Submission Checklist	44
11	**Conclusion**		**45–46**
		11.1 Summary of the Project	45
		11.2 Objectives Achieved	45
		11.3 Results Achieved	45
		11.4 Skills and Learning Outcomes	45
		11.5 Final Remarks	46
12	**References**		**47–48**
		12.1 Books	47
		12.2 Research Papers	47
		12.3 Linux / GNU Documentation	47
		12.4 Online Resources	48
	Appendix A: Placeholder Checklist		**49**
	Appendix B: Glossary		**50**

LIST OF FIGURES

No.	Title	Page
2.1	The fork–exec–wait model	5
5.1	Layered architecture of the Multi-Process Command Pipeline System	14
5.2	Workflow flowchart of the pipeline executor	15
5.3	Process and pipe topology for a four-stage pipeline	16
5.4	Descriptor table of a middle stage before and after dup2() and close()	17
5.5	Timeline showing overlapping execution of the stages	18
7.1	Successful compilation in Ubuntu terminal	29
7.2	Execution of the four-stage log-analysis pipeline with output	30
7.3	Execution report showing PIDs, exit statuses and elapsed time	30
7.4	Handling of an invalid command (unknowncommand | wc -l)	31
7.5	strace output showing pipe, clone, dup2, execve and wait4	31
10.1	Distribution of marks over the ten evaluation criteria	42

LIST OF TABLES

No.	Title	Page
1.1	Scope of the project	2
1.2	Objectives of the project	2
1.3	Target users and benefits	3
2.1	Comparison of IPC mechanisms	6
2.2	Existing approaches for chaining commands	7
2.3	Novelty and innovation compared with existing approaches	8
3.1	Hardware requirements	9
3.2	Software requirements	9
3.3	Functional requirements	10
3.4	Non-functional requirements	10
4.1	Common GCC options [[confirm project flags]]	11
4.2	Principal Linux system calls used	11
4.3	Linux utilities used	12
4.4	Other tools and libraries	13
5.1	Layers of the architecture	14
5.2	Workflow steps	15
5.3	Data transformation through the four stages (hypothetical illustration)	17
5.4	Module descriptions	18
5.5	Error conditions and behaviour	19
5.6	Key design decisions	19
6.1	fork() – summary	24
6.2	pipe() – summary	24
6.3	dup2() – summary	25
6.4	execvp() – summary	25
6.5	waitpid() – summary	26
6.6	close() – summary	26
6.7	Order of system calls for one pipeline stage	26
6.8	Error handling strategy	28
7.1	Demonstration input and output	30
8.1	Test environment	33
8.2	Functional test cases	33
8.3	Boundary and error-handling test cases	34
8.4	System-level verification tests	35
8.5	Summary of test results (to be completed)	36
8.6	Traceability matrix	36
8.7	Defect log (to be completed)	36
9.1	Results achieved	37
9.2	Measured execution results (to be completed)	37
9.3	Qualitative comparison	37
9.4	Business and application use cases	39
9.5	Stakeholder benefits	39

No.	Title	Page
9.6	SWOT – strengths and weaknesses	39
9.7	SWOT – opportunities and threats	40
9.8	Risks and mitigation	40
10.1	Mapping of the ten evaluation criteria	41
10.2	Team members and responsibilities (to be confirmed)	44
10.3	Pre-submission checklist	44
11.1	Objectives achieved	45
A.1	Placeholders to be completed	49
B.1	Glossary of terms	50

CHAPTER 1: INTRODUCTION

1.1 Background of the Project

Modern operating systems are built around a small number of powerful abstractions: the process, the file, and the channel that connects one process to another. In UNIX-like systems such as Linux, these abstractions are combined in a design philosophy that favours small programs, each doing one job well, whose output can be connected to the input of another program. The mechanism that makes this possible is the *pipe*, introduced in early UNIX and still the most widely used form of inter-process communication (IPC) on Linux today [1][9].

When a user types a command such as `cat log | grep ERROR | sort | uniq -c` in a shell, a considerable amount of operating-system work happens behind the scenes: the shell creates pipes, creates one process for every command, rewires the standard input and standard output of each process, replaces each process image with the requested program, and finally waits for all the processes to finish. For most users this machinery is invisible. For a student of operating systems, however, this hidden machinery is precisely what must be understood.

The project **"Multi-Process Command Pipeline System"** was developed for the Operating Systems and System Programming (OSSP) Hackathon. It is a C-based Linux application that accepts a pipeline of commands, executes every command as a separate process, connects the processes through pipes, and reports execution information such as process identifiers (PIDs), exit statuses and elapsed time. The project therefore turns an invisible shell feature into a transparent, inspectable and extensible program, and demonstrates the practical use of the Linux system calls `fork()`, `pipe()`, `dup2()`, `execvp()`, `waitpid()` and `close()`.

1.2 Problem Statement

Users frequently need the output of one Linux command to become the input of the next command. Achieving this correctly in a program, rather than at the shell prompt, is not trivial. The application must solve four related problems:

- **Executing multiple commands as one connected workflow** – several independent programs must behave as a single logical unit.
- **Managing communication between processes** – data produced by one process must flow reliably to the next through a kernel-managed channel.
- **Redirecting input and output correctly** – the standard streams of each process must be attached to the proper pipe ends without leaking descriptors.
- **Synchronizing children and collecting exit statuses** – the controlling process must wait for every child, avoid zombie processes and report how each stage terminated.

Without a dedicated mechanism, commands remain *disconnected*: each one runs alone, and the user must save intermediate results in temporary files or retype data by hand. The problem

addressed by this project is therefore to design and implement a **custom pipeline executor in C** that creates processes, connects commands using pipes, and reports execution information, while exposing the underlying operating-system concepts clearly.

1.3 Scope of the Project

The scope of the project is limited to the execution of linear pipelines of external Linux commands. It is intentionally a focused system-programming project and not a replacement for a full command shell. Table 1.1 summarises what lies inside and outside the scope.

Table 1.1: Scope of the project

Within Scope	Outside Scope (Limitations)
Execution of N commands (N ≥ 1) connected by pipes	A complete shell language (scripting, functions, job control)
Creation of N child processes and N − 1 pipes	Shell variables and command substitution [confirm against implementation]
Redirection of stdin/stdout using `dup2()`	Advanced redirection such as `2>&1`, here-documents and process substitution [confirm]
Program execution using `execvp()` with PATH search	Shell built-ins such as `cd` and `export` [confirm]
Waiting for children and collecting exit statuses using `waitpid()`	Background execution and job control
Report of PIDs, exit statuses and elapsed time	Graphical user interface
Testing on Ubuntu Linux / WSL	Non-POSIX platforms (native Windows, macOS not evaluated)

1.4 Objectives of the Project

The primary objective is to build a C application that executes a multi-stage command pipeline using Linux process-management and IPC system calls. The specific objectives, derived from the project proposal, are listed in Table 1.2.

Table 1.2: Objectives of the project

Obj.	Objective	Linux Feature / Deliverable
O1	Run multiple commands together as a single pipeline	Single pipeline executor
O2	Create one child process for each pipeline stage	`fork()`
O3	Connect adjacent processes through kernel pipes	`pipe()`
O4	Redirect standard input and output to pipe descriptors	`dup2()`
O5	Replace each child image with the requested command	`execvp()`
O6	Wait for children and collect their statuses	`waitpid()`
O7	Release unused descriptors so that EOF propagates	`close()`
O8	Display process IDs, exit statuses and elapsed time	Execution report

Obj.	Objective	Linux Feature / Deliverable
O9	Validate the system through systematic tests	Test suite (Chapter 8)

1.5 Importance of the Application

Pipelines are the backbone of text processing, log analysis, build automation and system administration on Linux. Understanding how a pipeline is realised by the kernel is a central learning outcome of any operating-systems course. The application is important for the following reasons:

- **Educational value** – it provides a compact, readable program in which each important OS concept appears in a few lines of code.
- **Transparency** – by reporting PIDs, exit statuses and elapsed time it makes visible information that a shell normally hides.
- **Foundation for tools** – the same technique underlies shells, job runners, build systems, CI agents and process supervisors.
- **Practice in robust system programming** – correct handling of descriptors, failures, EOF and zombies is a skill that transfers directly to professional systems work.

1.6 Target Users / Audience

Table 1.3: Target users and benefits

User Group	How They Benefit
Operating-systems students	Hands-on understanding of fork–exec–wait, pipes and descriptor redirection
Instructors and evaluators	A concrete reference implementation for laboratory demonstration and assessment
System administrators and DevOps engineers	A basis for lightweight command-orchestration utilities
Embedded and minimal-environment developers	A small C program that does not depend on a full shell
Open-source contributors	A clean code base on which new features can be built

1.7 Organization of the Report

The remaining chapters are organised as follows. Chapter 2 presents the operating-system concepts, existing approaches, the proposed approach and the novelty of the project. Chapter 3 lists hardware, software and functional requirements. Chapter 4 describes the technologies and Linux system calls. Chapter 5 presents the architecture and design with diagrams. Chapter 6 describes the implementation, algorithms, pseudocode and code. Chapter 7 explains compilation, execution and demonstration. Chapter 8 documents testing. Chapter 9 discusses results, limitations, future scope and business impact. Chapter 10 maps the work to the ten

hackathon evaluation criteria. Chapter 11 concludes the report and Chapter 12 lists the references.

Notation used in this report. Text highlighted in yellow and enclosed in square brackets, for example [to be filled], marks information that was not available in the project presentation (such as repository links, measured results, screenshots and names of faculty) and must be completed by the team. Appendix A collects all such placeholders. No results, screenshots or signatures have been invented.

CHAPTER 2: OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

2.1.1 Process and Process Identifier

A *process* is a program in execution together with its resources: an address space (text, data, heap and stack), a set of open file descriptors, credentials, and scheduling information stored by the kernel in a process control block. Every process is identified by a unique integer, the *process ID (PID)*. A process also has a parent PID (PPID), because every process except the first is created by another process [3][4]. In this project, the controlling program is the parent and each pipeline command runs in its own child process.

2.1.2 The fork–exec Model

UNIX separates process creation into two steps. `fork()` creates a new process that is an almost exact copy of the caller; `exec` then replaces the memory image of the calling process with a new program. Between these two steps the child can modify its own descriptor table, which is exactly how the shell, and this project, set up redirection. Modern kernels implement `fork()` with copy-on-write (COW) so that physical memory pages are copied only when one of the processes modifies them, which makes the fork–exec sequence efficient [1][2].

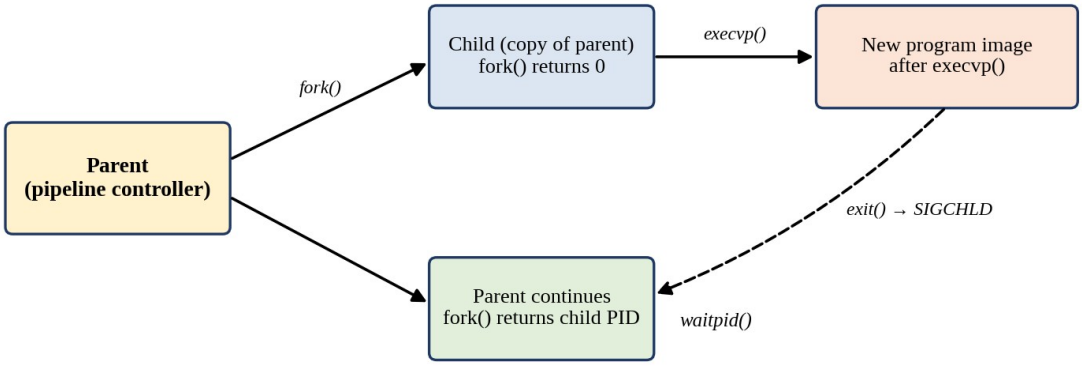


Figure 2.1: The fork–exec–wait model

2.1.3 File Descriptors and Open File Descriptions

A *file descriptor* is a small non-negative integer that indexes a per-process table. Each entry refers to an *open file description* maintained by the kernel. By convention descriptors 0, 1 and 2 are standard input (`STDIN_FILENO`), standard output (`STDOUT_FILENO`) and standard error (`STDERR_FILENO`). Programs such as `grep` and `sort` simply read from descriptor 0 and write to descriptor 1; they are unaware of whether these refer to a terminal, a file or a pipe. This uniformity is what allows `dup2()` to connect arbitrary programs without modifying them. Descriptors are inherited across `fork()` and preserved across `exec` unless the close-on-exec flag is set.

2.1.4 Pipes and Inter-Process Communication

A *pipe* is a unidirectional byte channel implemented by the kernel as a bounded buffer. `pipe(fd)` returns two descriptors: `fd[0]` for reading and `fd[1]` for writing. Data written to the write end is read from the read end in first-in first-out order. The important behaviours, documented in the Linux manual page `pipe(7)`, are [18]:

- The kernel buffer has a finite capacity (65,536 bytes by default on Linux). When it is full, a writer **blocks**; when it is empty, a reader **blocks**. This blocking behaviour provides implicit synchronization between producer and consumer.
- A `read()` returns 0 (end-of-file) only when **all** descriptors referring to the write end have been closed.
- A `write()` to a pipe whose read end is fully closed delivers `SIGPIPE` (or fails with `EPIPE`).
- Writes of at most `PIPE_BUF` bytes (4096 on Linux) are atomic.

Because EOF depends on every copy of the write end being closed, correct use of `close()` is the most common source of defects in pipeline programs; a single forgotten descriptor can make a downstream process such as `sort` wait forever.

Table 2.1: Comparison of IPC mechanisms

IPC Mechanism	Direction	Relationship	Suitability for This Project
Anonymous pipe	Unidirectional	Related processes (parent/child)	**Chosen** – ideal for linear pipelines
Named pipe (FIFO)	Unidirectional	Unrelated processes via filesystem name	Possible but needs filesystem entries
Message queue	Message-oriented	Any	Unnecessary structure and overhead
Shared memory	Bidirectional	Any	Requires explicit synchronization
Socket	Bidirectional	Local or network	Over-engineered for local stream
Temporary file	Sequential	Any	No concurrency, disk I/O, cleanup needed

2.1.5 Process Termination, Zombies and Synchronization

When a child terminates, the kernel keeps a small record (exit status) until the parent collects it with `wait()` or `waitpid()`. A terminated child whose status has not yet been collected is a *zombie*. A parent that never waits accumulates zombies, wasting process-table entries. The status returned by `waitpid()` encodes whether the child exited normally or was killed by a signal, and is decoded with the macros `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED` and `WTERMSIG`. The parent in this project calls `waitpid()` for every child, which synchronizes the end of the pipeline and supplies the exit-status information for the report.

2.1.6 Producer–Consumer Analogy

A pipeline is a classic instance of the producer–consumer problem with a bounded buffer. Each stage is a consumer of the previous pipe and a producer for the next. The kernel implements the bounded buffer and the necessary mutual exclusion and blocking, so the user program needs neither semaphores nor mutexes; correct descriptor management is sufficient [3].

2.2 Existing Methods / Algorithms

Several approaches exist for chaining commands programmatically. Their limitations motivate the proposed design.

Table 2.2: Existing approaches for chaining commands

Approach	Description	Limitations
Interactive shell pipeline (bash, dash)	Shell performs fork/pipe/dup2/exec internally	Mechanism hidden; no built-in consolidated report; a full shell is large and general-purpose
`system()`	Runs `/bin/sh -c "cmd"` and waits	Spawns an intermediate shell; no per-stage PID or status; shell-injection risk with untrusted input
`popen()`	Pipe to or from a single command	Unidirectional and for one command; cannot build an N-stage chain by itself
Temporary files	`cmd1 > t1; cmd2 < t1`	Sequential execution; disk I/O; large intermediates; cleanup required
Named pipes (FIFO)	Pipes with filesystem names	Extra setup and cleanup; not needed for related processes
Threads instead of processes	Run stages as threads sharing memory	Existing commands are separate executables; threads cannot `exec` them without a process

2.3 Proposed Approach

The proposed system implements the same sequence that a shell uses, but as a self-contained C program with explicit, documented stages. For a pipeline of N commands the program creates N − 1 pipes and N child processes. Each child connects its standard streams to the appropriate pipe ends with `dup2()`, closes every unused descriptor, and replaces itself with the requested program using `execvp()`. The parent closes the pipe ends it no longer needs, waits for every child with `waitpid()`, and finally prints an execution report. The overall flow, taken from the project proposal, is:

User input → Parse → pipe + fork → dup2 → execvp → waitpid → Output + report

2.4 Advantages of the Proposed Approach

- **No intermediate shell** – commands are executed directly through `execvp()`, avoiding the overhead and injection risks of `system()`.

- **Concurrent execution** – all stages run at the same time, and data streams through kernel buffers without temporary files.
- **Constant-memory streaming** – large data sets are processed without being held in memory by the controller.
- **Observability** – PIDs, exit statuses and elapsed time are collected and presented in one report.
- **Generality** – the same loop handles any number of stages.
- **Educational clarity** – every system call has a clearly identifiable purpose in the code.

2.5 Novelty and Innovation of the Project

The project does not claim to invent the pipe or to outperform existing shells. Its novelty lies in how the established mechanism is packaged, exposed and used for learning and tooling. The innovation points are summarised in the table below and are discussed against evaluation criterion 2 (Novelty and Innovation, 15 marks) in Chapter 10.

Table 2.3: Novelty and innovation compared with existing approaches

Aspect	Conventional Shell / Library Call	This Project
Visibility of OS mechanism	Hidden inside the shell	Each stage (pipe, fork, dup2, exec, wait) is explicit and inspectable
Execution information	Exit status of last command by default; per-stage status needs explicit user action	Consolidated report of PIDs, exit statuses and elapsed time for the pipeline [confirm exact report fields]
Dependence on a shell	`system()` and `popen()` start `/bin/sh`	Direct `execvp()`; no intermediate shell
Number of stages	Fixed by the code that calls `popen()`	Dynamic: N commands → N processes + N − 1 pipes
Failure handling	Message printed by shell	Failure of an individual stage is detected, reported, and does not leave the other stages blocked
Extensibility	Requires modifying a large shell	Small code base designed for adding logging, statistics and syntax

CHAPTER 3: SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The application is a lightweight command-line program and has modest hardware requirements. The values in Table 3.1 are recommended minimums for the development and demonstration environment and are not measured limits.

Table 3.1: Hardware requirements

Component	Minimum	Recommended
Processor	Any 64-bit x86-64 or ARM64 CPU	Dual-core or better (to observe true parallel execution of stages)
Memory (RAM)	2 GB	4 GB or more
Storage	500 MB free (OS tools, compiler, source)	2 GB free
Display / Input	Terminal and keyboard	Terminal with scroll-back for screenshots

3.2 Software Requirements

Table 3.2: Software requirements

Software	Purpose	Version Used
Ubuntu Linux or WSL 2 (Ubuntu)	Provides Linux process, pipe and descriptor interfaces	[Ubuntu version, e.g. 22.04 / 24.04]
GCC (GNU Compiler Collection)	Compiles the C source code	[gcc --version]
GNU Make	Automates compilation	[make --version]
Git and GitHub	Version control and project hosting	[git --version]
Text editor / IDE	Source editing	[e.g. VS Code, vim]
GDB, strace, Valgrind (optional)	Debugging and system-call tracing during testing	[confirm which were used]

3.3 Operating System Environment – Ubuntu / WSL

The program relies on POSIX interfaces implemented natively by the Linux kernel. Ubuntu was selected because it is widely used in education and ships GCC, GNU Make and the standard text-processing utilities (`cat`, `grep`, `sort`, `uniq`, `wc`) used in the demonstration. On Windows machines the Windows Subsystem for Linux 2 (WSL 2) runs a genuine Linux kernel and therefore supports `fork()`, `pipe()` and the other required calls without modification. Native Windows does not provide `fork()` and is therefore not supported.

3.4 Development Tools and Compilers

The code is compiled with GCC using warning flags so that errors such as unused variables or missing return values are caught at compile time. A typical command is shown below; the exact flags used by the project are defined in its Makefile [confirm].

Listing 3.1: Typical compilation command

```
gcc -std=c11 -Wall -Wextra -g -o pipeline src/*.c
```

3.5 Functional Requirements

Table 3.3: Functional requirements

ID	Requirement	Objective
FR1	The system shall accept a pipeline of one or more commands separated by the pipe symbol.	O1
FR2	The system shall split the input into commands and their arguments.	O1
FR3	The system shall create N − 1 pipes for N commands.	O3
FR4	The system shall create one child process per command.	O2
FR5	The system shall connect each child’s stdin/stdout to the correct pipe ends.	O4
FR6	The system shall execute each command using the PATH search.	O5
FR7	The system shall close all unused descriptors in parent and children.	O7
FR8	The system shall wait for all children and obtain their statuses.	O6
FR9	The system shall display PIDs, exit statuses and elapsed time.	O8
FR10	The system shall report an invalid or failing command without hanging.	O9

3.6 Non-Functional Requirements

Table 3.4: Non-functional requirements

Attribute	Requirement
Correctness	Output must be identical to that of the equivalent shell pipeline for supported syntax.
Reliability	No zombie processes, no descriptor leaks, no deadlock on EOF.
Performance	Stages run concurrently; the controller adds negligible overhead compared with the commands themselves.
Portability	Runs on any POSIX-compliant Linux with GCC.
Maintainability	Modular code, meaningful names, comments and a Makefile.
Usability	Clear error messages and a readable execution report.

CHAPTER 4: TECHNOLOGY AND TOOLS USED

4.1 C Programming Language

C was selected because it provides direct, low-overhead access to the operating-system interface. The Linux system calls are specified in terms of C types (`pid_t`, `int`, `char *const []`) and C is the language in which the kernel and most system utilities are written [6]. The language offers explicit control over memory and descriptors, which is essential when the learning objective is to see exactly what the operating system is doing. The project uses standard C together with POSIX headers (`unistd.h`, `sys/types.h`, `sys/wait.h`, `stdlib.h`, `stdio.h`, `string.h`, `errno.h`).

4.2 GCC Compiler

The GNU Compiler Collection (GCC) translates the C source into an executable ELF binary for Linux [21]. Useful options are summarised in Table 4.1.

Table 4.1: Common GCC options [confirm project flags]

Option	Meaning
`-std=c11`	Compile according to the C11 standard
`-Wall -Wextra`	Enable most warnings to catch mistakes early
`-g`	Include debugging symbols for `gdb` and `valgrind`
`-O2`	Optimize for the final build (optional)
`-o name`	Name of the output executable

4.3 Linux System Calls

A *system call* is the controlled entry point through which a user program requests a service from the kernel. The C library (glibc) provides wrapper functions such as `fork()` and `pipe()` that place arguments in registers and execute the special instruction that transfers control to the kernel. Table 4.2 lists the system calls central to this project; their detailed behaviour is explained in Section 6.4.

Table 4.2: Principal Linux system calls used

Call	Header	Prototype	Role in the Project
`fork()`	unistd.h	`pid_t fork(void)`	Creates a child process for each pipeline stage
`pipe()`	unistd.h	`int pipe(int fd[2])`	Creates a unidirectional channel between two stages
`dup2()`	unistd.h	`int dup2(int old, int new)`	Redirects stdin/stdout to pipe ends
`execvp()`	unistd.h	`int execvp(const char *file, char *const argv[])`	Replaces the child with the requested command

Call	Header	Prototype	Role in the Project
`waitpid()`	sys/wait.h	`pid_t waitpid(pid_t pid, int *st, int opt)`	Waits for a child and retrieves its status
`close()`	unistd.h	`int close(int fd)`	Releases unused descriptors so that EOF propagates

In addition, the program uses the library function `clock_gettime()` (with `CLOCK_MONOTONIC`) for elapsed-time measurement [confirm the timing method used in the implementation], `_exit()` for terminating a child after a failed `execvp()`, and `perror()` / `strerror()` for diagnostics.

4.4 POSIX Threads / Synchronization

The project does **not** use POSIX threads. Each command must be executed as a separate program, which requires a separate process image created by `fork()` and `execvp()`; threads share a single address space and cannot run an independent executable. Synchronization is achieved without explicit locks: (i) the kernel's blocking pipe semantics synchronize each producer–consumer pair, and (ii) `waitpid()` synchronizes the parent with the termination of every child. Because no shared user-space data is modified concurrently, no mutexes or semaphores are required.

4.5 Linux Commands and Utilities

Table 4.3: Linux utilities used

Utility	Use in the Project
`cat`	Reads a file and writes it to standard output (stage 1 of the demonstration)
`grep`	Filters lines containing a pattern, e.g. ERROR
`sort`	Sorts lines; reads all input until EOF before writing – a good test of EOF propagation
`uniq -c`	Collapses adjacent identical lines and prefixes counts
`wc`	Counts lines, words or bytes
`printf`	Generates controlled test input
`ps`, `pstree`, `lsof`, `/proc`	Inspecting processes and descriptors during demonstration
`strace`	Tracing system calls (fork/clone, pipe, dup2, execve, wait4) [if used]
`valgrind`	Detecting memory errors and leaks [if used]

4.6 Other Tools / Libraries Used

Table 4.4: Other tools and libraries

Tool	Purpose
GNU Make	Builds the project with a single `make` command and tracks dependencies [22]
Git	Distributed version control of the source code [23]
GitHub	Hosts the repository, README, issues and commit history [24] – URL: [GitHub repository URL]
Standard C library (glibc)	Provides wrappers for system calls and string/I/O functions [19]

Listing 4.1: Illustrative Makefile (replace with the project's own Makefile)

```
# Illustrative Makefile
CC      = gcc
CFLAGS  = -std=c11 -Wall -Wextra -g
TARGET  = pipeline
SRCS    = $(wildcard src/*.c)

all: $(TARGET)

$(TARGET): $(SRCS)
      $(CC) $(CFLAGS) -o $@ $^

clean:
      rm -f $(TARGET)

.PHONY: all clean
```

CHAPTER 5: SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

The system follows a layered design. The user interacts with the application layer, which consists of five cooperating components. The application layer requests services only through the system-call interface, and the Linux kernel supplies process scheduling, pipe buffers and file-descriptor tables. Figure 5.1 shows this structure.

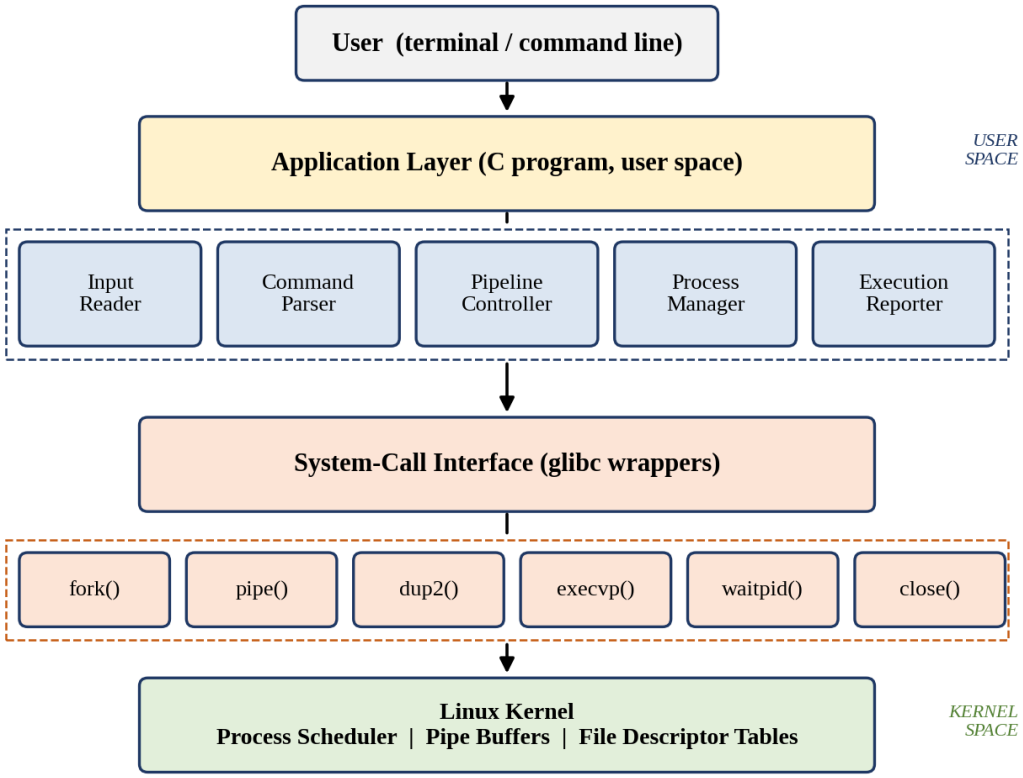


Figure 5.1: Layered architecture of the Multi-Process Command Pipeline System

Table 5.1: Layers of the architecture

Layer	Components	Responsibility
User	Terminal / command line	Supplies the pipeline string and views output and report
Application (user space)	Input Reader, Command Parser, Pipeline Controller, Process Manager, Execution Reporter	Implements the pipeline logic described in Chapter 6
System-call interface	`fork`, `pipe`, `dup2`, `execvp`, `waitpid`, `close`	Controlled gateway to kernel services
Kernel (kernel space)	Scheduler, pipe buffers, descriptor tables	Executes processes, buffers data between them

The textual form of the architecture in the project presentation is: *User Command → Command Parser → Stage Identification → Pipe Creation → fork() per stage → dup2() redirection → execvp() execution → waitpid() synchronization → Execution Report*. For N commands the system uses **N child processes and N − 1 pipes**; each pipe links the standard output of one process to the standard input of the next.

5.2 System Workflow

The workflow in Figure 5.2 expands the architecture into a step-by-step control flow. The loop over the stages creates a pipe (except after the last stage), forks a child, and then sets up the child and the parent differently according to the value returned by `fork()`.

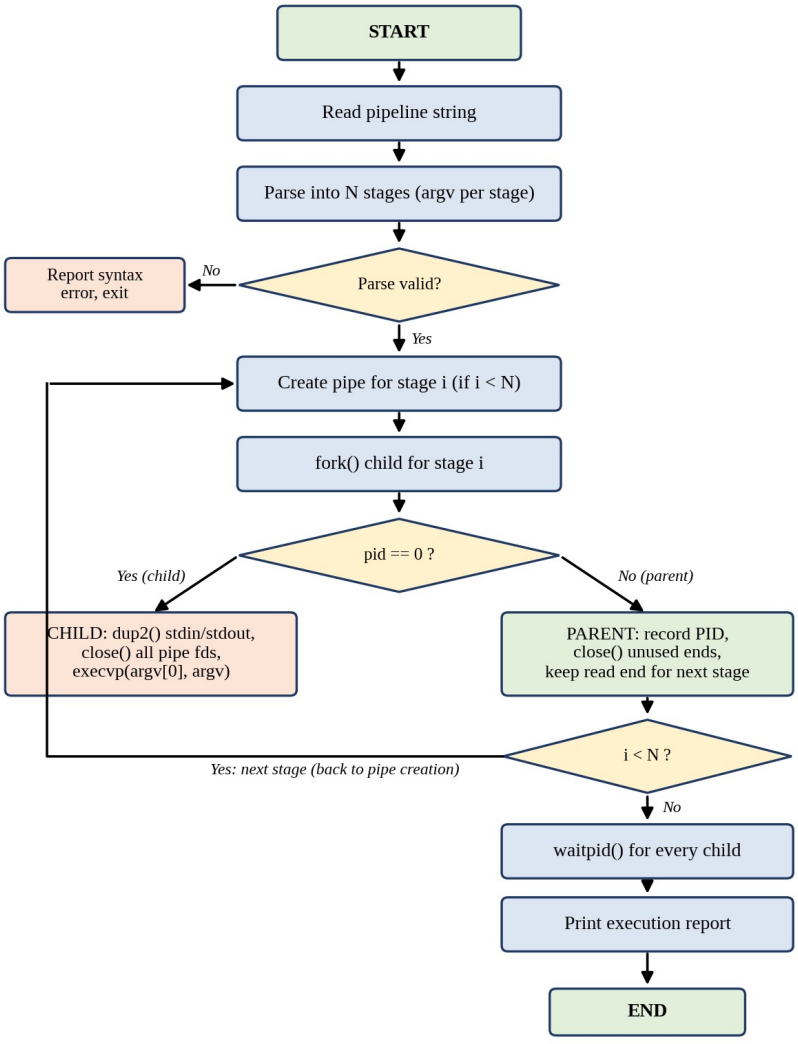


Figure 5.2: Workflow flowchart of the pipeline executor

Table 5.2: Workflow steps

Step	Action	System Call / Function
1	Read the pipeline entered by the user	`fgets()` / command-line argument [confirm input method]

Step	Action	System Call / Function
2	Parse commands and arguments into argument vectors	String functions
3	Create a pipe between adjacent commands	`pipe()`
4	Create the child process for the stage	`fork()`
5	Connect stdin/stdout to the pipe descriptors	`dup2()`
6	Execute the command in the child	`execvp()`
7	Close unused pipe descriptors in parent and children	`close()`
8	Wait for all processes and collect statuses	`waitpid()`
9	Display execution information	`fprintf()`

5.3 Flowchart / Data Flow

5.3.1 Process and Pipe Topology

Figure 5.3 shows the processes and pipes created for the four-stage demonstration pipeline `cat | grep ERROR | sort | uniq -c`. The parent creates all four children, but the children communicate directly with each other through the kernel pipes; the parent is not on the data path.

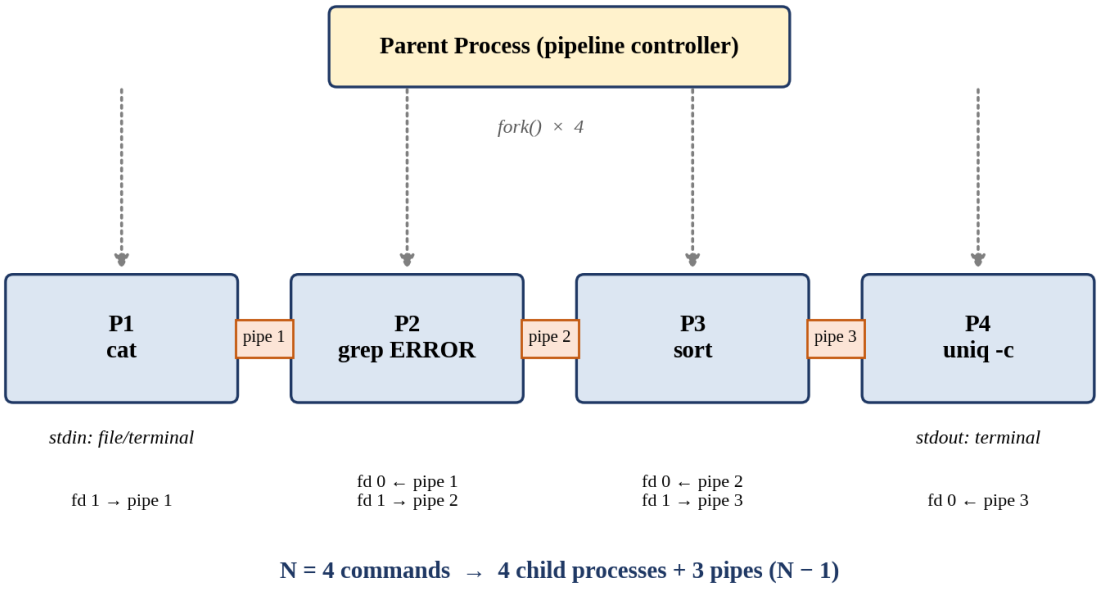


Figure 5.3: Process and pipe topology for a four-stage pipeline

5.3.2 Descriptor Table Transformation

Figure 5.4 illustrates how the descriptor table of a middle stage changes. After `fork()` the child holds copies of the parent's descriptors, including both ends of the pipes. `dup2()` then installs the correct pipe ends at descriptors 0 and 1, and `close()` removes the now-redundant originals. Failing to close the extra write ends would prevent the downstream reader from ever seeing EOF.

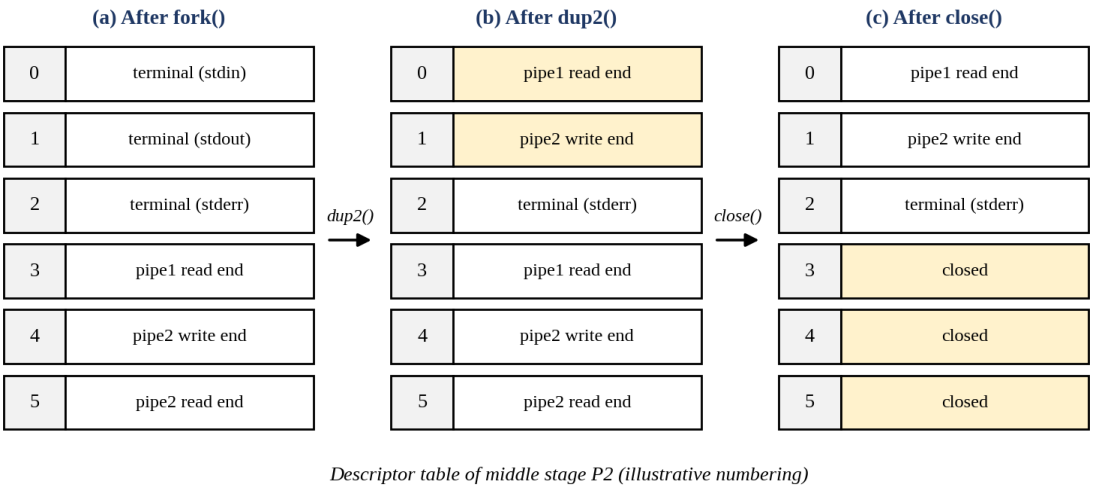


Figure 5.4: Descriptor table of a middle stage before and after dup2() and close()

5.3.3 Data Transformation Through the Stages

Table 5.3 traces data through the demonstration pipeline using a small **hypothetical** input chosen only to explain the stages; it is not the project's test data. The final output is consistent with the expected output given in the project presentation.

Table 5.3: Data transformation through the four stages (hypothetical illustration)

Stage	Command	Data Leaving the Stage
Input (hypothetical file)	—	ERROR Database connection failed INFO Service started ERROR Network timeout ERROR Database connection failed
P1	`cat`	All four lines, unchanged
P2	`grep ERROR`	The three lines containing ERROR
P3	`sort`	The two "Database" lines adjacent, followed by the "Network" line
P4	`uniq -c`	2 ERROR Database connection failed 1 ERROR Network timeout

5.3.4 Concurrency of the Stages

Although the stages are connected in a chain, they execute concurrently. Figure 5.5 shows the time relationship between the parent and the children. A downstream stage may begin as soon as upstream data is available; stages such as `sort` must wait for EOF before producing output.

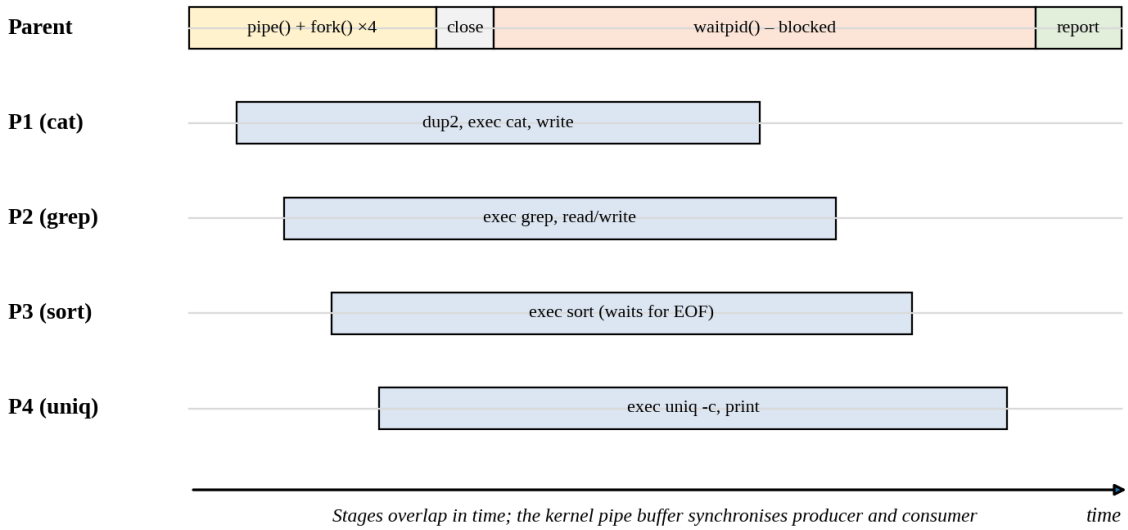


Figure 5.5: Timeline showing overlapping execution of the stages

5.4 Module Description

Table 5.4: Module descriptions

Module	Responsibility	Main Inputs → Outputs
Input Reader	Obtains the pipeline line from the user	Keyboard / argument → string
Command Parser	Splits the line at pipe symbols; tokenizes each command into an argument vector terminated by NULL; validates empty stages	String → array of stages
Pipeline Controller	Determines N, creates N − 1 pipes, drives the stage loop	Stages → pipe descriptors
Process Manager	Forks children; in each child performs `dup2()`, `close()` and `execvp()`; in parent closes unused ends and records PIDs	Stage + descriptors → running processes
Synchronizer	Calls `waitpid()` for every child and decodes the status	PIDs → exit statuses
Execution Reporter	Prints PIDs, exit statuses and elapsed time	Statuses + timestamps → report

5.5 Input and Output Design

5.5.1 Input

The input is a single line containing one or more commands separated by the vertical bar. Example: `cat data/sample.log | grep ERROR | sort | uniq -c`. The method of supplying the line (interactive prompt or command-line argument) and the supported quoting rules are [to be confirmed from the implementation]. The test cases in the project presentation

use quoted arguments such as `printf "hello\n"`, so quoted-string handling in the parser is expected.

5.5.2 Output

The output consists of (a) the normal output of the last command, written to the terminal, and (b) the execution report. The report presents, for every stage, the PID and the exit status, together with the elapsed time. A suggested layout is shown below; the actual layout is [replace with the implementation's real report format].

Listing 5.1: Suggested layout of the execution report

```
--- Execution report ---
Stage 1  cat   PID <pid>  exit status <n>
Stage 2  grep  PID <pid>  exit status <n>
Stage 3  sort  PID <pid>  exit status <n>
Stage 4  uniq  PID <pid>  exit status <n>
Elapsed time: <seconds>
```

5.5.3 Exit Codes and Diagnostics

Table 5.5: Error conditions and behaviour

Situation	Behaviour
Empty or malformed pipeline	Syntax error message; no process created
`pipe()` or `fork()` failure	Message printed with `perror()`; pipeline aborted
Command not found	Child prints a diagnostic and exits with status 127 (shell convention) [confirm]
Command found but not executable	Child exits with status 126 (shell convention) [confirm]
Child terminated by a signal	Report states the signal number

5.6 Design Decisions and Rationale

Table 5.6: Key design decisions

Decision	Rationale
Use `execvp()` rather than `system()`	Avoids an intermediate shell, gives the true PID of each command and removes shell-injection risk
Create pipes one at a time inside the loop	Needs only three descriptors alive at any moment instead of $2(N - 1)$
Close descriptors in both parent and child	Guarantees EOF propagation and prevents descriptor leaks
Wait for every child	Prevents zombies and supplies per-stage status
Use processes, not threads	Each command is a separate executable
Child exits with `_exit()` after failed `execvp()`	Prevents the child from flushing duplicated stdio buffers or continuing the parent's loop

CHAPTER 6: IMPLEMENTATION

Implementation note. The project presentation specifies the implementation approach (Slide 6) and shows one illustrative fragment. The listings in this chapter apply the same approach in complete, compilable form. They were written to explain the technique and have been checked to compile with GCC and to follow standard Linux semantics, but they are a **reference implementation**: function names and structure in the team's repository may differ. Before submission the team must replace or align the listings with the actual repository source [verify listings against GitHub repository].

6.1 Module-wise Implementation

The implementation follows the nine steps listed in the project presentation: read the pipeline; parse commands and arguments; create pipes between adjacent commands; create the required child processes; connect stdin/stdout to pipe descriptors; execute the commands; close unused descriptors; wait for processes and collect statuses; and display execution information. The fragment below, reproduced from the presentation, shows the essence of one pipeline stage in the child process.

Listing 6.1: Illustrative stage fragment from the project presentation (not the complete implementation)

```
// child process (one pipeline stage)
pipe(fd);
pid_t pid = fork();
if (pid == 0) {
  dup2(fd[1], STDOUT_FILENO);
  close(fd[0]);
  close(fd[1]);
  execvp(args[0], args);
}
```

The data structure used to describe one stage is a record holding the argument vector, the PID of the child and the raw status returned by `waitpid()`.

Listing 6.2: Stage descriptor (reference implementation)

```
typedef struct {
    char  *argv[MAX_ARGS];  /* NULL-terminated argument vector */
    pid_t  pid;             /* PID of the child running this stage */
    int    status;          /* raw status from waitpid() */
} Stage;
```

6.2 Algorithm / Pseudocode

6.2.1 Algorithm 1 – Parse Pipeline

Listing 6.3: Pseudocode for parsing

```
INPUT : line (string)
OUTPUT: stages[0..N-1], each with argv[], or ERROR

N <- 0
for each segment in split(line, "|"):
    tokens <- tokenize(segment)        // whitespace; quotes if supported
    if tokens is empty: return ERROR   // e.g. "a | | b" or trailing "|"
    stages[N].argv <- tokens + [NULL]
    N <- N + 1
if N = 0: return ERROR
return N
```

6.2.2 Algorithm 2 – Execute Pipeline

Listing 6.4: Pseudocode for pipeline execution

```
INPUT : stages[0..N-1]
prev_read <- -1                        // read end of previous pipe
start_time <- now()
for i <- 0 to N-1:
    if i < N-1: fd <- pipe()           // create pipe for stage i
    pid <- fork()
    if pid = 0:                        // child
        CHILD_SETUP(i, prev_read, fd)
    else:                              // parent
        stages[i].pid <- pid
        if prev_read != -1: close(prev_read)
        if i < N-1: close(fd[1])       // parent never writes
        prev_read <- fd[0] (or -1 after last stage)
for i <- 0 to N-1:
    waitpid(stages[i].pid, &status)
    stages[i].status <- status
end_time <- now()
PRINT_REPORT(stages, end_time - start_time)
```

6.2.3 Algorithm 3 – Child Setup

Listing 6.5: Pseudocode for child-process setup

```
procedure CHILD_SETUP(i, prev_read, fd):
    if prev_read != -1:
        dup2(prev_read, STDIN)         // read from previous stage
        close(prev_read)
    if i < N-1:
        dup2(fd[1], STDOUT)            // write to next stage
        close(fd[0]); close(fd[1])
    execvp(argv[0], argv)              // only returns on error
    print error message
    _exit(127 if not found else 126)
```

6.2.4 Algorithm 4 – Wait and Report

Listing 6.6: Pseudocode for synchronization and reporting

```
for each stage s:
    if WIFEXITED(s.status):   report exit status WEXITSTATUS(s.status)
    elif WIFSIGNALED(s.status): report signal WTERMSIG(s.status)
report elapsed time
```

Complexity. Parsing is O(L) in the length L of the input line. Execution creates O(N) processes and O(N) pipes; the controller performs O(N) system calls for creation and O(N) for waiting. Memory used by the controller is O(N) for the stage table, independent of the amount of data flowing through the pipes.

6.3 Core C / C++ Implementation

Listings 6.7 to 6.10 present the reference implementation in four parts: headers and the parser, stage creation, synchronization and reporting, and the main function.

Listing 6.7: Headers and parser (reference implementation; no quote handling)

```
#define _POSIX_C_SOURCE 200809L
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <errno.h>
#include <time.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_ARGS 64

/* Split "line" on '|' and whitespace. Returns stage count or -1. */
static int parse_pipeline(char *line, Stage *st, int max)
{
    int n = 0;
    char *seg = line, *next;
    do {
        next = strchr(seg, '|');
        if (next) *next++ = '\0';       /* terminate this segment */
        if (n >= max) return -1;
        int a = 0;
        char *save = NULL;
        for (char *tok = strtok_r(seg, " \t\n", &save); tok;
             tok = strtok_r(NULL, " \t\n", &save)) {
            if (a >= MAX_ARGS - 1) return -1;
            st[n].argv[a++] = tok;
        }
        if (a == 0) return -1;           /* empty stage: "a | | b", "a |" */
        st[n].argv[a] = NULL;
        n++;
        seg = next;
    } while (seg);
    return n;
}
```

Listing 6.8: Stage creation loop (reference implementation)

```
static int run_pipeline(Stage *st, int n)
{
    int prev_read = -1;              /* read end of previous pipe */
    struct timespec t0, t1;
    clock_gettime(CLOCK_MONOTONIC, &t0);

    for (int i = 0; i < n; i++) {
        int fd[2] = {-1, -1};
        if (i < n - 1 && pipe(fd) == -1) { perror("pipe"); return -1; }

        pid_t pid = fork();
        if (pid == -1) { perror("fork"); return -1; }

        if (pid == 0) {              /* ---- child: stage i ---- */
            if (prev_read != -1) {
                if (dup2(prev_read, STDIN_FILENO) == -1) _exit(126);
                close(prev_read);
            }
            if (fd[1] != -1) {
                if (dup2(fd[1], STDOUT_FILENO) == -1) _exit(126);
                close(fd[0]);
                close(fd[1]);
            }
            execvp(st[i].argv[0], st[i].argv);
            fprintf(stderr, "pipeline: %s: %s\n",
                    st[i].argv[0], strerror(errno));
            _exit(errno == ENOENT ? 127 : 126);
        }
        /* ---- parent ---- */
        st[i].pid = pid;
        if (prev_read != -1) close(prev_read);
        if (fd[1] != -1) close(fd[1]);
        prev_read = fd[0];
    }
    if (prev_read != -1) close(prev_read);
    /* ... waiting and reporting: see Listing 6.9 ... */
```

Listing 6.9: Synchronization and report (end of run_pipeline)

```
    for (int i = 0; i < n; i++) {
        while (waitpid(st[i].pid, &st[i].status, 0) == -1) {
            if (errno != EINTR) { perror("waitpid"); break; }
        }
    }
    clock_gettime(CLOCK_MONOTONIC, &t1);

    fprintf(stderr, "\n--- Execution report ---\n");
    for (int i = 0; i < n; i++) {
        fprintf(stderr, "Stage %d  %-10s PID %-7d ", i + 1,
                st[i].argv[0], (int)st[i].pid);
        if (WIFEXITED(st[i].status))
            fprintf(stderr, "exit status %d\n", WEXITSTATUS(st[i].status));
        else if (WIFSIGNALED(st[i].status))
            fprintf(stderr, "killed by signal %d\n",
                    WTERMSIG(st[i].status));
    }
```

```
    fprintf(stderr, "Elapsed time: %.6f s\n", elapsed(&t0, &t1));
    return 0;
}
```

Listing 6.10: Main function (reference implementation)

```
int main(int argc, char *argv[])
{
    char buf[4096];
    if (argc == 2) {
        strncpy(buf, argv[1], sizeof buf - 1);
        buf[sizeof buf - 1] = 0;
    } else {
        printf("pipeline> ");
        if (!fgets(buf, sizeof buf, stdin)) return 0;
    }
    Stage st[32];
    memset(st, 0, sizeof st);
    int n = parse_pipeline(buf, st, 32);
    if (n < 1) { fprintf(stderr, "pipeline: syntax error\n"); return 2; }
    return run_pipeline(st, n) == 0 ? 0 : 1;
}
```

6.4 Linux System Calls / APIs Used

This section explains each required system call in the context of the project. The descriptions follow the Linux manual pages and POSIX specification [12]–[18][20].

6.4.1 fork() – Creating a Process for Each Stage

Table 6.1: fork() – summary

Attribute	Description
Prototype	`pid_t fork(void);` (header `<unistd.h>`).
Purpose	Creates a new process (the child) as a duplicate of the calling process (the parent). The child receives copies of the parent's memory (copy-on-write), descriptor table and environment, but has its own PID.
Return value	In the parent, the PID of the child; in the child, 0; on failure, −1 with `errno` set (for example `EAGAIN` when the process limit is reached, or `ENOMEM`).
Use in the project	Called once per pipeline command inside the stage loop; the return value is used to decide whether the code that follows is executed by the child (set up redirection and `exec`) or by the parent (record PID and continue).
Pitfall	Because the child inherits unflushed stdio buffers, the child must leave through `execvp()` or `_exit()` and not through `exit()`.

6.4.2 pipe() – Creating the Communication Channel

Table 6.2: pipe() – summary

Attribute	Description
Prototype	`int pipe(int fd[2]);` (header `<unistd.h>`).

Attribute	Description
Purpose	Creates a unidirectional kernel buffer and returns two descriptors: `fd[0]` is the read end and `fd[1]` the write end.
Return value	0 on success, −1 on error (for example `EMFILE` when the per-process descriptor limit is reached).
Use in the project	One pipe is created for every adjacent pair of commands, i.e. N − 1 pipes. The pipe must be created **before** `fork()` so that both the writer and the reader inherit the descriptors.
Pitfall	Unclosed descriptors leak and suppress EOF. On x86-64 glibc the wrapper may invoke the `pipe` or `pipe2` kernel entry point; this is visible in `strace` output.

6.4.3 dup2() – Redirecting Standard Streams

Table 6.3: dup2() – summary

Attribute	Description
Prototype	`int dup2(int oldfd, int newfd);` (header `<unistd.h>`).
Purpose	Makes `newfd` refer to the same open file description as `oldfd`; if `newfd` was already open it is silently closed first, and this close-and-copy is performed atomically. If `oldfd` equals `newfd`, nothing is done.
Return value	`newfd` on success, −1 on error.
Use in the project	In a child, `dup2(prev_read, STDIN_FILENO)` attaches the previous pipe to standard input, and `dup2(fd[1], STDOUT_FILENO)` attaches the next pipe to standard output. After the call, the original pipe descriptors are redundant and are closed. This is the call that allows unmodified programs to take part in a pipeline.

6.4.4 execvp() – Executing the Command

Table 6.4: execvp() – summary

Attribute	Description
Prototype	`int execvp(const char *file, char *const argv[]);` (header `<unistd.h>`).
Purpose	Replaces the current process image with a new program. The “v” indicates that arguments are supplied as a vector (a NULL-terminated array) and the “p” indicates that, if `file` contains no slash, the directories in the `PATH` environment variable are searched.
Return value	It does not return on success; it returns −1 only on failure, with `errno` set (`ENOENT` if the file is not found, `EACCES` if it is not executable).
Use in the project	Each child calls `execvp(argv[0], argv)` with the stage’s argument vector. The PID is preserved by `exec`, so the PID recorded by the parent is the PID of the command itself. Open descriptors (including the redirected 0 and 1) remain open in the new program.
Pitfall	The code following `execvp()` must handle failure and call `_exit()`.

6.4.5 waitpid() – Synchronization and Exit Status

Table 6.5: waitpid() – summary

Attribute	Description
Prototype	`pid_t waitpid(pid_t pid, int *wstatus, int options);` (header `<sys/wait.h>`).
Purpose	Suspends the caller until the specified child changes state (normally, terminates) and retrieves its status. With `pid` greater than 0 it waits for that particular child; options such as `WNOHANG` allow non-blocking checks.
Return value	The PID of the child whose state changed; −1 on error (`ECHILD` if there is no such child, `EINTR` if interrupted by a signal).
Use in the project	The parent calls `waitpid()` for every recorded PID. The status is decoded with `WIFEXITED()` / `WEXITSTATUS()` for normal termination and `WIFSIGNALED()` / `WTERMSIG()` for termination by signal. Reaping every child prevents zombies and provides the data for the execution report.

6.4.6 close() – Releasing Descriptors

Table 6.6: close() – summary

Attribute	Description
Prototype	`int close(int fd);` (header `<unistd.h>`).
Purpose	Deletes a descriptor from the process's table. The underlying open file description, and in the case of a pipe the corresponding pipe end, is released only when **all** descriptors referring to it (in all processes) have been closed.
Return value	0 on success, −1 on error.
Use in the project	(i) each child closes the original pipe descriptors after `dup2()`; (ii) the parent closes the write end immediately after forking the writer and the read end after forking the reader. Only when every copy of a write end is closed does the downstream reader receive EOF, which allows commands such as `sort` and `wc` to finish.

Table 6.7: Order of system calls for one pipeline stage

Order	Process	Call	Effect
1	Parent	`pipe(fd)`	Pipe i created (two descriptors)
2	Parent	`fork()`	Child i created; inherits both descriptors
3	Child i	`dup2(prev_read, 0)`	stdin ← previous pipe
4	Child i	`dup2(fd[1], 1)`	stdout → next pipe
5	Child i	`close(...)`	Original descriptors released
6	Child i	`execvp(argv[0], argv)`	Process becomes the command
7	Parent	`close(prev_read)`, `close(fd[1])`	Parent keeps only `fd[0]` for the next stage
8	Parent	`waitpid(pid_i)` (loop)	Statuses collected; zombies avoided

6.5 Process / Thread Implementation

For a pipeline of N commands the system runs N + 1 processes: the parent controller and N children. No threads are used. All children are created before the parent begins waiting, so they execute concurrently and the kernel scheduler distributes them over the available CPU cores. The process tree for the demonstration pipeline is:

Listing 6.11: Process tree for the demonstration pipeline

```
pipeline            (parent controller)
 |-- cat            (stage 1)
 |-- grep ERROR     (stage 2)
 |-- sort           (stage 3)
 `-- uniq -c        (stage 4)
```

Each child is a direct child of the controller, so the controller can wait for each by PID. If the parent terminated before its children they would become orphans adopted by `init`/`systemd`; the controller therefore always waits before exiting.

6.6 IPC / Synchronization Implementation

Pipes are the only IPC mechanism. Correct behaviour depends on four rules, each of which is enforced in the implementation:

1. **Create before fork.** Each pipe is created before the stage that writes to it is forked so that both ends are inherited.
2. **Close what you do not use.** Each process closes every pipe end it does not need. This ensures that EOF is delivered when the upstream process exits.
3. **Never let the parent hold a write end.** The parent closes `fd[1]` immediately after forking the writer; otherwise downstream stages never see EOF and the pipeline hangs.
4. **Wait for every child.** The parent reaps each PID so that exit statuses are collected and zombies do not accumulate.

The kernel’s bounded buffer provides back-pressure: when a fast producer fills the 64 KiB buffer, it blocks until the consumer reads. Deadlock cannot occur in a linear pipeline in which every process reads from at most one pipe and writes to at most one pipe, provided the unused ends are closed. If a downstream process terminates early (for example `head`), the upstream writer receives `SIGPIPE` and terminates; this is normal behaviour and is reported as termination by signal 13.

6.7 Memory Management Implementation

The reference implementation stores the stage table on the stack and each argument vector as pointers into the input buffer, so no dynamic allocation is needed for typical inputs. If the project allocates memory dynamically (for example for an unbounded number of stages or

arguments), every allocation must be released after `waitpid()` completes and the code should be validated with Valgrind [state whether dynamic allocation is used in the project]. Memory behaviour specific to process creation is:

- **Copy-on-write:** `fork()` does not copy physical pages immediately; they are shared until written.
- **Replacement of image:** `execvp()` discards the child's copy of the parent's memory and loads the new program, so any memory allocated by the parent before the fork is not retained by the command.
- **Descriptor table:** each process has its own table; a missed `close()` is a resource leak limited by `ulimit -n`.
- **Buffer limits:** the input buffer and the maximum number of arguments are fixed constants, which prevents buffer overruns by length checks in the parser.

6.8 Optional User Interface Implementation

The application uses a command-line interface. The user enters (or passes) the pipeline string and sees the command output followed by the execution report. No graphical interface is implemented, as the focus of the project is operating-system functionality. A possible enhancement is an interactive prompt with history and a verbose mode that prints each system call as it is made (see Future Scope, Section 9.5).

6.9 Error Handling Implementation

Table 6.8: Error handling strategy

Possible Failure	Detection	Handling
Empty stage / malformed input	Parser returns −1	Print syntax error, create no processes
`pipe()` fails	Return value −1	Print `perror()`, abort; close open descriptors
`fork()` fails	Return value −1	Print `perror()`, abort; wait for children already created
`dup2()` fails in child	Return value −1	Child exits with 126
`execvp()` fails	Returns (never on success)	Child prints message with `strerror(errno)`, exits 127/126
`waitpid()` interrupted	`errno == EINTR`	Retry
Child killed by signal	`WIFSIGNALED`	Reported with signal number

CHAPTER 7: EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled on Ubuntu Linux (or WSL 2) from the project directory. If a Makefile is provided, compilation is a single command; otherwise GCC is invoked directly. Replace the executable and source names with those of the repository [confirm executable name and source layout].

Listing 7.1: Compilation commands

```
$ cd pipeline
$ make                      # using the project Makefile
# or, directly:
$ gcc -std=c11 -Wall -Wextra -g -o pipeline src/*.c
```

A successful build produces the executable without warnings. Any warning should be investigated, since warnings frequently indicate descriptor or return-value mistakes.

[SCREENSHOT PLACEHOLDER: paste a real screenshot of the make / gcc output]

Figure 7.1: Successful compilation in Ubuntu terminal

7.2 Execution Commands

Listing 7.2: Execution commands (confirm the invocation style used by the project)

```
# interactive prompt form (confirm which form the project uses)
$ ./pipeline
pipeline> cat data/sample.log | grep ERROR | sort | uniq -c

# argument form
$ ./pipeline "cat data/sample.log | grep ERROR | sort | uniq -c"
```

7.3 Sample Inputs and Outputs

The principal demonstration of the project presentation is the four-stage log-analysis pipeline. The expected output given in the presentation is reproduced in Table 7.1. The “Actual Output” column must be completed from a real run.

Table 7.1: Demonstration input and output

Item	Content
Input pipeline	`cat data/sample.log | grep ERROR | sort | uniq -c`
Input file	`data/sample.log` – [attach/describe the actual file]
Expected output (from presentation)	2 ERROR Database connection failed 1 ERROR Network timeout
Actual output	[ ]
Execution report (PIDs, exit statuses, elapsed time)	[paste real report from a run]

Explanation of stages. `cat` reads the sample log; `grep ERROR` keeps lines containing ERROR; `sort` arranges the matching lines so that identical lines are adjacent; `uniq -c` counts consecutive identical lines. The application creates the four processes, connects their stdin/stdout through pipes, and reports process IDs, exit statuses and elapsed time.

7.4 Screenshots of Execution

[SCREENSHOT PLACEHOLDER: paste real screenshot (Slide 7 of the presentation also requires one)]

Figure 7.2: Execution of the four-stage log-analysis pipeline with output

[SCREENSHOT PLACEHOLDER: paste real screenshot]

Figure 7.3: Execution report showing PIDs, exit statuses and elapsed time

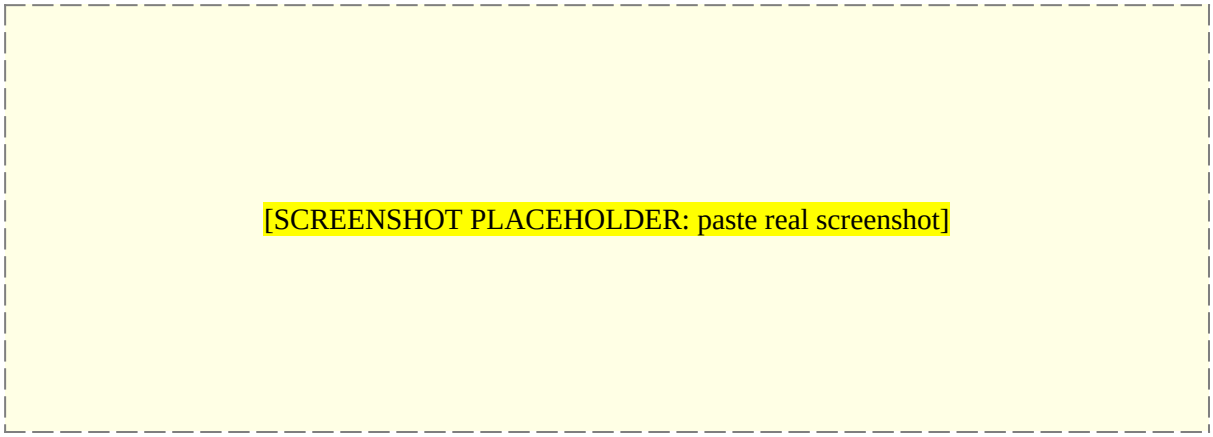


Figure 7.4: Handling of an invalid command (unknowncommand | wc -l)

7.5 Demonstration of OS Concepts

The following commands let the evaluator observe the operating-system concepts directly. The outputs depend on the machine and must be captured by the team.

7.5.1 Tracing the System Calls

Listing 7.3: Tracing the system calls with strace

```
$ strace -f -e trace=pipe,pipe2,clone,clone3,fork,dup2,dup3,execve,wait4,close \
        ./pipeline "cat data/sample.log | grep ERROR | sort | uniq -c"
```

The trace should show, for every stage, a `pipe`/`pipe2` call, a `clone` (the kernel entry used by glibc's `fork()`), `dup2` (or `dup3` on some architectures), `close` calls, `execve` (the kernel entry used by `execvp()`) and finally `wait4` (used by `waitpid()`) in the parent.

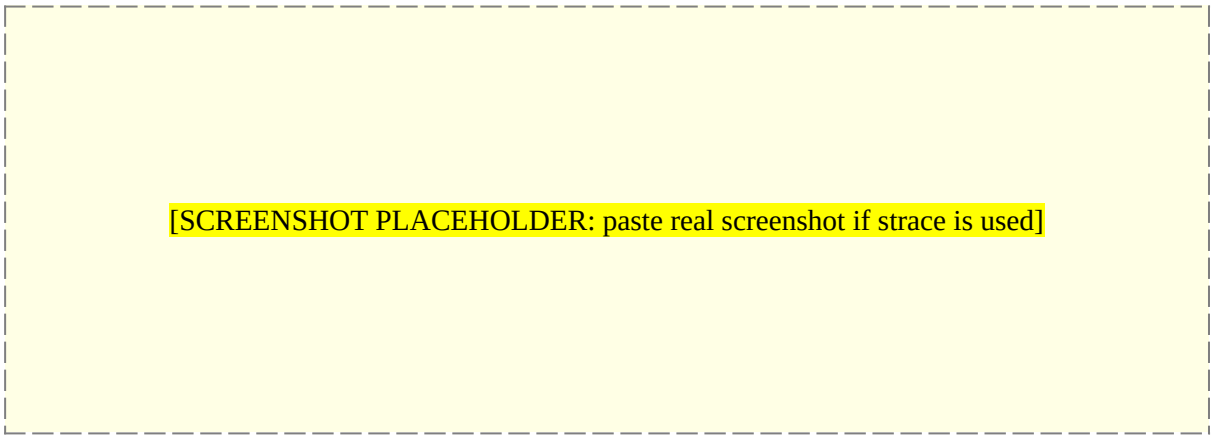


Figure 7.5: strace output showing pipe, clone, dup2, execve and wait4

7.5.2 Observing Processes and Descriptors

Listing 7.4: Observing the process tree and pipe descriptors while the pipeline runs

```
# terminal 1
$ ./pipeline "sleep 30 | sleep 30 | sleep 30"

# terminal 2 (while terminal 1 is still running)
```

```
$ pstree -p $(pgrep -n pipeline)
$ ls -l /proc/<PID of a sleep>/fd
```

The first command shows the parent with its children. The second shows that descriptors 0 and/or 1 of the child processes refer to `pipe:[inode]` objects, confirming the effect of `dup2()`. Matching inode numbers in adjacent stages show that the two processes share the same pipe.

7.5.3 Verifying that No Zombies Remain

Listing 7.5: Checking for zombie processes

```
$ ./pipeline "echo hello | wc -c"
$ ps -o pid,stat,cmd --ppid $$ | grep " Z"      # expect no output
```

CHAPTER 8: TESTING AND VALIDATION

8.1 Testing Strategy

Testing verifies command execution, multi-stage communication, output correctness and error handling. Four levels of testing are applied:

- **Functional testing** – each supported pipeline produces the output that the equivalent shell pipeline would produce.
- **Boundary and error testing** – malformed input, missing commands and failing stages are handled without hanging.
- **System-level OS testing** – no zombies, no descriptor leaks, correct EOF propagation, correct signal reporting.
- **Regression testing** – the full suite is re-run after every change in the repository.

The expected result of each test is determined from the documented behaviour of the Linux utilities, i.e. by comparison with the output of the same pipeline in `bash`. The **Actual** and **Status** columns (highlighted) are intentionally left as placeholders and must be completed after the test run on the project's code.

8.2 Test Environment

Table 8.1: Test environment

Item	Value
Operating system	[Ubuntu version / WSL 2]
Kernel	[uname -r]
Compiler	[gcc --version]
Build command	[make / gcc ...]
Date of testing	[dd-mm-yyyy]
Tester(s)	[names]

8.3 Functional Test Cases

Test cases T01–T05 are taken from the project presentation (Slide 8). Test cases T06–T10 extend the suite to cover single commands, options, many stages and large data.

Table 8.2: Functional test cases

ID	Test Case	Input	Expected Result	Actual	Status
T01	Basic execution	`printf "hello\n"`	Displays hello	[ ]	[ ]
T02	Sorting	`printf "c\na\nb\n" | sort`	Displays a, b, c on separate lines	[ ]	[ ]
T03	Word counting	`printf "hello world\n" | wc -w`	Displays 2	[ ]	[ ]

ID	Test Case	Input	Expected Result	Actual	Status
T04	Log analysis	`cat data/sample.log | grep ERROR | sort | uniq -c`	Counts matching error entries (expected in presentation: 2 Database connection failed; 1 Network timeout)	[ ]	[ ]
T05	Invalid command	`unknowncommand | wc -l`	Reports command failure for stage 1; `wc -l` receives EOF and prints 0	[ ]	[ ]
T06	Single command	`echo hello`	Displays hello; exactly one child process	[ ]	[ ]
T07	Two stages with option	`echo hello | tr a-z A-Z`	Displays HELLO	[ ]	[ ]
T08	Three stages	`printf "b\na\nb\n" | sort | uniq`	Displays a then b	[ ]	[ ]
T09	Many stages (10)	`echo x | cat | cat | cat | cat | cat | cat | cat | cat | cat`	Displays x; 10 children created	[ ]	[ ]
T10	Large data (exceeds 64 KiB pipe buffer)	`seq 1 200000 | wc -l`	Displays 200000; no deadlock	[ ]	[ ]

8.4 Boundary and Error-Handling Test Cases

Table 8.3: Boundary and error-handling test cases

ID	Test Case	Input	Expected Result	Actual	Status
T11	Empty input	(empty line)	Syntax error; no process created	[ ]	[ ]
T12	Trailing pipe	`ls |`	Syntax error; no process created	[ ]	[ ]
T13	Empty middle stage	`ls | | wc -l`	Syntax error; no process created	[ ]	[ ]
T14	Failing middle stage	`echo hi | unknowncommand | wc -l`	Failure of stage 2 reported (status 127 if shell convention is followed [confirm]); `wc -l` prints 0; no hang	[ ]	[ ]
T15	Missing input file	`cat missing.txt | wc -l`	`cat` prints error on stderr and exits with status 1; `wc -l` prints 0	[ ]	[ ]
T16	Exit-status propagation	`false | true`	Report shows stage 1 status 1 and stage 2 status 0	[ ]	[ ]
T17	Early-terminating consumer	`yes | head -n 3`	Displays y three times; `yes` reported as terminated by signal 13 (SIGPIPE); program does not hang	[ ]	[ ]
T18	Too many arguments / stages	Input exceeding configured limits	Rejected with message; no buffer overflow [confirm limits]	[ ]	[ ]

ID	Test Case	Input	Expected Result	Actual	Status
T19	Quoted argument	`printf "a b\n" | wc -w`	Displays 2 if quoting is supported [confirm parser capability]	[ ]	[ ]

8.5 System-Level (OS) Verification Tests

Table 8.4: System-level verification tests

ID	Property	Method	Expected Result	Actual	Status
S01	No zombie processes	Run any pipeline; inspect `ps` for defunct children	No child in state Z	[ ]	[ ]
S02	Distinct PIDs	Inspect execution report	N distinct PIDs for N stages	[ ]	[ ]
S03	Concurrency	Run `sleep 2 | sleep 2 | sleep 2` and observe elapsed time	About 2 s (stages overlap), not 6 s	[ ]	[ ]
S04	Pipe descriptors in children	`ls -l /proc/<pid>/fd` while running	fd 0/1 refer to `pipe:[...]`	[ ]	[ ]
S05	No descriptor leak	`valgrind --track-fds=yes ./pipeline "..."`	Only descriptors 0, 1, 2 open at exit	[ ]	[ ]
S06	No memory leak	`valgrind --leak-check=full`	No definitely-lost blocks	[ ]	[ ]
S07	System calls present	`strace -f` trace	pipe, clone, dup2, execve, wait4 per stage	[ ]	[ ]

8.6 Automated Test Harness

A small shell script can run the cases automatically and report the totals needed for Slide 8 of the presentation (“Passed / Failed / Total”). The script below is illustrative; it was checked against a reference build and must be adapted to the project’s invocation style [confirm].

Listing 8.1: Illustrative automated test script

```
#!/bin/bash
# run_tests.sh - illustrative test harness
PASS=0; FAIL=0
check() {   # check "description" "pipeline" "expected output"
  local desc="$1" cmd="$2" expected="$3" actual
  actual=$(./pipeline "$cmd" 2>/dev/null)
  if [ "$actual" == "$expected" ]; then
    echo "PASS: $desc"; PASS=$((PASS+1))
  else
    echo "FAIL: $desc"; FAIL=$((FAIL+1))
  fi
}
check "single command" "echo hello" "hello"
check "two stages"     "echo hello | tr a-z A-Z" "HELLO"
check "word count"     "echo hello world | wc -w" "2"
check "large data"     "seq 1 200000 | wc -l" "200000"
echo "Passed: $PASS  Failed: $FAIL  Total: $((PASS+FAIL))"
```

8.7 Test Summary

Table 8.5: Summary of test results (to be completed)

Category	Total	Passed	Failed
Functional (T01–T10)	10	[__]	[__]
Boundary / error (T11–T19)	9	[__]	[__]
System-level (S01–S07)	7	[__]	[__]
Overall	26	[__]	[__]

8.8 Traceability: Objectives to Tests

Table 8.6: Traceability matrix

Objective	Verified by
O1 Single pipeline	T04, T07–T09
O2 `fork()` per stage	T06, T09, S02, S07
O3 `pipe()` connection	T02, T04, T10, S04
O4 `dup2()` redirection	T02, T03, S04
O5 `execvp()` execution	T01, T05, T14
O6 `waitpid()` status	T16, T17, S01
O7 `close()` and EOF	T02, T10, T17, S05
O8 Execution report	T04, T16, S02, S03
O9 Validation	T01–T19, S01–S07

8.9 Defect Log

Defects found during testing should be recorded with their cause and fix. Typical defects in pipeline programs include an unclosed write end (pipeline hangs), a missed `wait` (zombie) and a child that continues to run parent code after a failed `execvp()`.

Table 8.7: Defect log (to be completed)

Defect ID	Description	Cause	Fix	Status
D1	[ ]	[ ]	[ ]	[ ]
D2	[ ]	[ ]	[ ]	[ ]

CHAPTER 9: RESULTS, LIMITATIONS, FUTURE SCOPE AND BUSINESS IMPACT

9.1 Results Achieved

According to the project presentation (Slide 9), the following results were achieved. Each result should be supported by evidence (screenshots, test logs, repository links) before final submission.

Table 9.1: Results achieved

Result	Linked Objective	Evidence
Multi-process command execution	O1, O2, O5	[Test T01–T10 log / screenshot]
Inter-process communication through pipes	O3, O7	[Test T02, T10 / strace]
Input/output redirection using `dup2()`	O4	[Test S04 / /proc listing]
Reports process execution information (PIDs, exit statuses, elapsed time)	O6, O8	[Report screenshot]
Foundation for learning Linux process management	—	This report; presentation; repository

9.2 Measured Results

Quantitative measurements (elapsed time and exit statuses) must be recorded from real runs. No numerical results are claimed in this report.

Table 9.2: Measured execution results (to be completed)

Pipeline	Stages	Elapsed Time	Exit Statuses
`cat | grep | sort | uniq -c`	4	[ ]	[ ]
`seq 1 200000 | wc -l`	2	[ ]	[ ]
`sleep 2 | sleep 2 | sleep 2`	3	[ ] (expected ≈ 2 s if concurrent)	[ ]

9.3 Comparison with Alternative Approaches

Table 9.3: Qualitative comparison

Feature	Shell pipeline	`system()`	Temp files	This project
Runs stages concurrently	Yes	No (one command string via shell)	No	Yes
Avoids intermediate shell	No (is the shell)	No	No	Yes

Feature	Shell pipeline	`system()`	Temp files	This project
Per-stage PID and status in one report	Needs `PIPESTATUS` / manual work	No	Manual	Yes
Needs disk for intermediates	No	No	Yes	No
Visible OS mechanism for teaching	Hidden	Hidden	Hidden	Explicit
Full scripting language	Yes	Via shell	Via shell	No (focused tool)

9.4 Limitations

The project presentation states the following limitations, which are retained here:

- The system is a **custom pipeline executor, not a complete shell**.
- **Quoting and syntax support depend on the parser**; unusual quoting and escape sequences may not be handled.
- **Shell variables, command substitution and advanced redirection** may not be supported.

In addition, because commands are executed directly, shell built-ins (for example `cd`) and shell-specific syntax are unavailable unless implemented [confirm].

9.5 Future Scope

- Better parser validation and more informative error messages.
- More automated test cases, integrated into continuous integration.
- Execution statistics and configurable logging (per-stage CPU time using `wait4()` resource usage; JSON/CSV export).
- Additional command syntax: file redirection (`<`, `>`, `>>`), background execution (`&`), quoting and escaping.
- Improved resource management and error handling, including signal handling (`SIGINT` forwarding) and process groups.
- An optional verbose mode that prints the system calls as they are made, for classroom demonstration.

9.6 Business Impact

Although developed as an academic hackathon project, the system addresses a practical need and has measurable value in education and in lightweight automation. This section evaluates the business impact (evaluation criterion 10, 5 marks) qualitatively; no market data, cost savings or user numbers are claimed, since none were measured.

9.6.1 Value Proposition

- **Reduced training cost** – a small, understandable code base shortens the time required to teach process management and IPC.
- **Lower operational overhead** – a direct `execvp()` design avoids launching a shell per command and is suitable for minimal environments.
- **No licensing cost** – built entirely on open-source tools (GCC, GNU Make, Git, Linux).
- **Reusable asset** – the pipeline engine can be embedded in larger utilities.

9.6.2 Use Cases

Table 9.4: Business and application use cases

Sector	Use Case	Benefit
Education	Laboratory demonstrations of fork–exec–pipe–wait	Clear mapping from theory to code
DevOps / Administration	Log analysis pipelines with execution reports	Auditable per-stage status
Embedded / minimal Linux	Chaining utilities without a full shell	Small footprint
Software testing / CI	Orchestrating command sequences and collecting exit codes	Reliable failure detection
Security tooling	Executing commands without a shell interpreter	Reduced injection surface

9.6.3 Stakeholder Benefits

Table 9.5: Stakeholder benefits

Stakeholder	Benefit
Students	Practical mastery of core OS concepts
Institution / faculty	A reusable teaching and assessment example
Developers / operators	A base for command-orchestration tools
Open-source community	A small, extensible project to build on

9.6.4 SWOT Analysis

Table 9.6: SWOT – strengths and weaknesses

Strengths	Weaknesses
Direct use of OS primitives; concurrent; transparent; low resource use; open source	Not a full shell; limited syntax; parser capability [confirm]

Table 9.7: SWOT – opportunities and threats

Opportunities	Threats
Extension with redirection, logging and statistics; use in teaching; integration into CI tooling	Mature shells already cover most functionality; platform dependency on POSIX systems

9.6.5 Risks and Mitigation

Table 9.8: Risks and mitigation

Risk	Mitigation
Descriptor leaks causing hangs	Systematic `close()` discipline and `valgrind --track-fds` tests
Zombie accumulation	Always `waitpid()` every child
Unsafe input handling	Length limits, no shell invocation, validation in parser
Platform limitations (no `fork()` on native Windows)	Document WSL 2 / Linux requirement

CHAPTER 10: HACKATHON EVALUATION CRITERIA MAPPING

The OSSP Hackathon is evaluated on ten criteria carrying a total of 100 marks. Table 10.1 maps each criterion to the way it is addressed in this project and to the sections of this report. Figure 10.1 shows the distribution of marks. The “Status” column indicates what remains for the team to complete; no marks are claimed.

Table 10.1: Mapping of the ten evaluation criteria

No.	Criterion	Marks	How Addressed	Report Sections	Status
1	Problem Definition	10	Clear statement of the need to connect commands, redirect I/O and synchronize processes	1.2, 1.3, 1.4	Complete
2	Novelty and Innovation	15	Shell-independent executor with transparent mechanism and consolidated execution report	2.5, 10.2	Complete [confirm report fields]
3	Architecture Design	10	Layered architecture, workflow, topology and descriptor diagrams	Chapter 5	Complete
4	Implementation	20	Use of fork, pipe, dup2, execvp, waitpid, close; algorithms and code	Chapter 6	[verify against repository]
5	Testing	10	26 designed test cases with expected results; harness	Chapter 8	[run tests; fill actual]
6	Documentation	10	This report in university format; README	All; 10.6	Complete after placeholders
7	GitHub Repository	5	Version-controlled source, Makefile, README, tests	10.7	[URL; commits]
8	Presentation	10	Ten-slide deck with demo	10.8	[add real screenshots]
9	Teamwork	5	Defined roles and collaboration	10.9	[confirm contributions]
10	Business Impact	5	Use cases, stakeholders, SWOT, risks	9.6	Complete

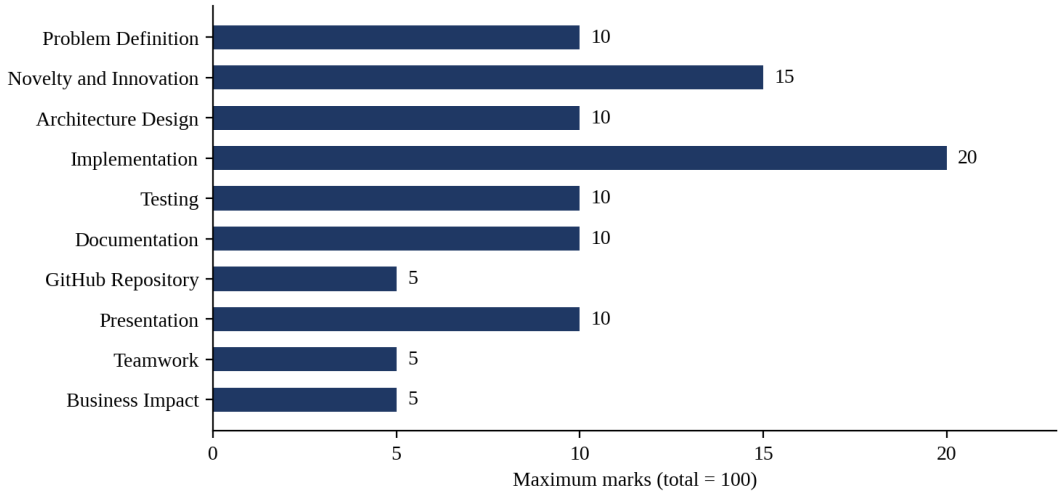


Figure 10.1: Distribution of marks over the ten evaluation criteria

10.1 Criterion 1 – Problem Definition (10 Marks)

The problem is defined precisely in Section 1.2: the need to execute multiple commands as one connected workflow, manage communication between processes, redirect input and output correctly, and synchronize children while collecting exit statuses. Section 1.3 bounds the scope, Section 1.4 states measurable objectives and Table 1.2 links each objective to a Linux feature, so that every objective can later be verified by a test (Section 8.8).

10.2 Criterion 2 – Novelty and Innovation (15 Marks)

The innovation is not a new IPC mechanism but a new **packaging and visibility** of an existing one. (i) The program is a shell-independent executor that invokes commands directly through `execvp()`. (ii) It generalises to any number of stages (N processes, N − 1 pipes). (iii) It produces a consolidated execution report with process IDs, exit statuses and elapsed time. (iv) It makes each operating-system call explicit and observable, which turns an invisible shell feature into a teaching and tooling asset. Table in Section 2.5 contrasts these points with the conventional approaches, and the Future Scope outlines further innovations (statistics, logging, extended syntax). The report is candid that shells already provide the pipe feature itself.

10.3 Criterion 3 – Architecture Design (10 Marks)

Chapter 5 presents a layered architecture (Figure 5.1), the workflow flowchart (Figure 5.2), the process and pipe topology (Figure 5.3), the descriptor-table transformation (Figure 5.4) and a concurrency timeline (Figure 5.5), followed by module descriptions, input/output design, error conditions and recorded design decisions with rationale.

10.4 Criterion 4 – Implementation (20 Marks)

Chapter 6 details the implementation: algorithms and pseudocode, the stage loop, the explanation of all six required system calls with prototypes, return values and pitfalls, process

and IPC implementation, memory management and error handling. The highest-weighted criterion therefore receives the most detailed treatment. Before submission the listings must be aligned with the repository code [verify].

10.5 Criterion 5 – Testing (10 Marks)

Chapter 8 defines 26 tests in three categories (functional, boundary/error, system-level), each with input and expected result, an automated harness, a traceability matrix and a defect log. The actual results are to be filled after execution; the results summary table is ready for the counts required by the presentation.

10.6 Criterion 6 – Documentation (10 Marks)

This document is prepared in a professional university-report format following the department's sample: cover page, declaration, certificate, acknowledgement, abstract, index, list of figures and tables, twelve chapters, references and appendices. In addition to the report, the repository should contain a README (see below).

10.7 Criterion 7 – GitHub Repository (5 Marks)

The repository URL is [https://github.com/…]. A well-organised repository is expected to follow the structure below, with a README explaining purpose, build and run instructions, sample output, team members and licence; regular, meaningful commits by all members; and tagged releases.

Listing 10.1: Recommended repository layout

```
pipeline/
├── README.md          # purpose, build, usage, examples, team
├── Makefile
├── src/               # C source files
├── data/sample.log    # sample input used in the demo
├── tests/run_tests.sh # automated tests
├── docs/              # this report, presentation, screenshots
└── LICENSE
```

10.8 Criterion 8 – Presentation (10 Marks)

The ten-slide presentation covers: title and team; problem statement; objectives and proposed solution; system architecture; technologies and OS concepts; implementation approach; working demonstration; testing and validation; results, limitations and future scope; and conclusion with team contributions. Two items must be completed before presenting: replace the placeholder on the demonstration slide with a real screenshot, and replace the "Passed / Failed / Total" placeholders on the testing slide with real counts.

10.9 Criterion 9 – Teamwork (5 Marks)

Roles were proposed in the presentation and are reproduced in Table 10.2. They must be updated if the actual contributions differ. Evidence of teamwork (commit history by each member, pull requests, meeting notes) should be referenced here [add evidence].

Table 10.2: Team members and responsibilities (to be confirmed)

Member	Roll No.	Proposed Responsibility	Evidence of Contribution
Sindhuja	2520080045	System architecture and process-management concepts	[commits / sections]
Jhanasri Thota	2520090121	C implementation and pipeline execution	[commits / sections]
Alekhya	2520090126	Testing, documentation and validation	[commits / sections]

10.10 Criterion 10 – Business Impact (5 Marks)

Section 9.6 identifies the value proposition, use cases across education, DevOps, embedded systems, testing and security tooling, stakeholder benefits, SWOT analysis and risk mitigation, without unverified numerical claims.

10.11 Pre-Submission Checklist

Table 10.3: Pre-submission checklist

Item	Done?
All yellow placeholders filled (Appendix A)	[ ]
Listings match the repository	[ ]
Tests run; actual results and summary filled	[ ]
Screenshots inserted (Chapter 7)	[ ]
Repository public, README complete, commits from all members	[ ]
Signatures obtained on certificate (guide, coordinator, HOD)	[ ]
Index and list of figures/tables updated (Word: right-click → update)	[ ]

CHAPTER 11: CONCLUSION

11.1 Summary of the Project

The Multi-Process Command Pipeline System is a C-based Linux application that executes a pipeline of commands as a set of cooperating processes. For N commands it creates N child processes and N − 1 pipes, redirects each child's standard streams with `dup2()`, runs each command with `execvp()`, closes unused descriptors with `close()`, and synchronizes with the children using `waitpid()` before printing an execution report of process IDs, exit statuses and elapsed time. The system demonstrates fundamental operating-system concepts through a practical application: process creation, inter-process communication, I/O redirection, command execution and process synchronization.

11.2 Objectives Achieved

Table 11.1: Objectives achieved

Objective	Status
O1 Run multiple commands as one pipeline	Achieved (as per presentation) [confirm with tests]
O2–O7 Use fork, pipe, dup2, execvp, waitpid, close correctly	Achieved (as per presentation) [confirm with tests]
O8 Report PIDs, exit statuses and elapsed time	Achieved (as per presentation) [attach evidence]
O9 Validate through tests	Test suite designed; execution results [pending]

11.3 Results Achieved

Multi-process command execution, inter-process communication through pipes, input/output redirection and the reporting of process execution information were achieved, and the project provides a foundation for learning Linux process management. Quantitative results are to be added from actual runs (Table 9.2).

11.4 Skills and Learning Outcomes

- Understanding of process creation, the fork–exec model and the process life cycle.
- Ability to use pipes and file-descriptor redirection for inter-process communication.
- Practice in system-call error handling, signal behaviour and zombie avoidance.
- Design and documentation of a modular C program with Makefile-based builds.
- Use of Git and GitHub for collaborative development.
- Test design, including expected-versus-actual verification and OS-level checks.
- Teamwork, planning and technical communication.

11.5 Final Remarks

The project shows that a small program, built directly on a handful of system calls, can reproduce the core of a shell’s pipeline machinery while making it observable. Its limitations are clear and its future scope is concrete, making it a sound base for further work in system programming.

CHAPTER 12: REFERENCES

Online resources were consulted for the manual pages and documentation listed below; access dates are [add access dates].

12.1 Books

[1] M. Kerrisk, *The Linux Programming Interface*. San Francisco, CA, USA: No Starch Press, 2010.

[2] W. R. Stevens and S. A. Rago, *Advanced Programming in the UNIX Environment*, 3rd ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2013.

[3] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018.

[4] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA, USA: Pearson, 2015.

[5] R. Love, *Linux System Programming*, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2013.

[6] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Englewood Cliffs, NJ, USA: Prentice Hall, 1988.

[7] R. E. Bryant and D. R. O'Hallaron, *Computer Systems: A Programmer's Perspective*, 3rd ed. Boston, MA, USA: Pearson, 2016.

[8] E. S. Raymond, *The Art of UNIX Programming*. Boston, MA, USA: Addison-Wesley, 2003.

12.2 Research Papers

[9] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974.

[10] D. M. Ritchie, "The evolution of the UNIX time-sharing system," *AT&T Bell Laboratories Technical Journal*, vol. 63, no. 8, pp. 1577–1593, 1984.

[11] A. Baumann, J. Appavoo, O. Krieger and T. Roscoe, "A fork() in the road," in *Proc. 17th Workshop on Hot Topics in Operating Systems (HotOS)*, Bertinoro, Italy, 2019, pp. 14–22.

12.3 Linux / GNU Documentation

[12] Linux man-pages project, "fork(2) – create a child process." https://man7.org/linux/man-pages/man2/fork.2.html

[13] Linux man-pages project, "pipe(2) – create pipe." https://man7.org/linux/man-pages/man2/pipe.2.html

[14] Linux man-pages project, "dup(2) – duplicate a file descriptor (dup2)." https://man7.org/linux/man-pages/man2/dup.2.html

[15] Linux man-pages project, “exec(3) – execute a file (execvp).” https://man7.org/linux/man-pages/man3/exec.3.html

[16] Linux man-pages project, “wait(2) – wait for process to change state (waitpid).” https://man7.org/linux/man-pages/man2/wait.2.html

[17] Linux man-pages project, “close(2) – close a file descriptor.” https://man7.org/linux/man-pages/man2/close.2.html

[18] Linux man-pages project, “pipe(7) – overview of pipes and FIFOs.” https://man7.org/linux/man-pages/man7/pipe.7.html

[19] GNU Project, “The GNU C Library Reference Manual.” https://www.gnu.org/software/libc/manual/

[20] The Open Group, “POSIX.1-2017 (IEEE Std 1003.1-2017) Base Specifications, Issue 7.” https://pubs.opengroup.org/onlinepubs/9699919799/

[21] GNU Project, “GCC, the GNU Compiler Collection – Online Documentation.” https://gcc.gnu.org/onlinedocs/

[22] GNU Project, “GNU Make Manual.” https://www.gnu.org/software/make/manual/

12.4 Online Resources

[23] Git SCM, “Git Documentation.” https://git-scm.com/doc

[24] GitHub, “GitHub Docs.” https://docs.github.com

[25] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, “Operating Systems: Three Easy Pieces – Process API and Pipes.” https://pages.cs.wisc.edu/~remzi/OSTEP/

[26] Project presentation, “Multi-Process Command Pipeline System,” OS & System Programming Hackathon, 2026. Repository: [GitHub URL].

APPENDIX A: PLACEHOLDER CHECKLIST

The following information was not available in the provided files and is marked with yellow-highlighted placeholders. Search the document for “[” to locate each one.

Table A.1: Placeholders to be completed

No.	Item to Complete	Where
1	Guide name, designation; course coordinator; HOD; academic year; date and place	Cover, Declaration, Certificate, Acknowledgement
2	Confirmation of the department name on the cover	Cover
3	Student signatures (declaration) and faculty signatures (certificate)	Declaration, Certificate
4	GitHub repository URL, commit history, README confirmation	4.6, 10.7, References
5	Software versions (Ubuntu, GCC, make, git, kernel)	3.2, 8.2
6	Invocation style, quoting support, input limits, report format	5.5, 7.2, Chapter 8
7	Actual file/function names; verification of listings against source	Chapter 6
8	Contents of `data/sample.log` and real demonstration output	7.3
9	All screenshots	Chapter 7
10	Actual test results, pass/fail counts, defect log, test date and testers	Chapter 8
11	Measured execution times and exit statuses	9.2
12	Evidence of team contributions	10.9
13	Access dates of online references	Chapter 12

APPENDIX B: GLOSSARY

Table B.1: Glossary of terms

Term	Meaning
PID	Process identifier, a unique integer for each process
File descriptor	Integer index into a process's table of open files
Pipe	Kernel-managed unidirectional byte stream between processes
EOF	End of file; a read returns 0 when all write ends are closed
Zombie	Terminated child whose status has not yet been collected
Orphan	Process whose parent has terminated; adopted by init/systemd
COW	Copy-on-write; pages copied only when modified
SIGPIPE	Signal sent when writing to a pipe with no readers
IPC	Inter-process communication
WSL	Windows Subsystem for Linux
OSSP	Operating Systems and System Programming